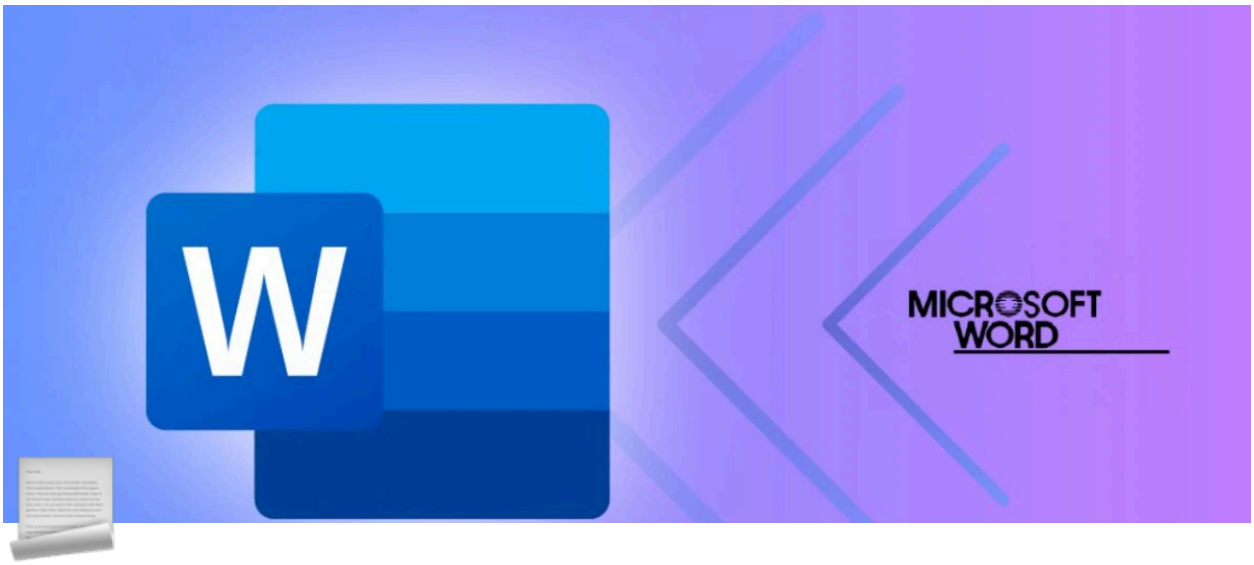

Metin Okuma

- **<stdio.h>** içinde bulunan FILE struct'ı içinde bulunur.
- İmleçi hareket ettiriyormuş gibi düşün.

Fonksiyonlar:

```
FILE
fopen("dosya_yolu_ve_adı","dosya_modu") Dosya açmak açmak ve oluşturmak
için kullanılır.
fclose()   Dosya kapatmak için. En sona yazılmalıdır.
feof()     Dosyanın sonuna gelindiğini sorgulamak için.
fprintf()  Dosyaya formatlı yazmak için.
fscanf()   Dosyadan formatlı okumak için.
fputs()    Dosyaya katar yazmak için.
fgets()
fwrite()   Göremediğimiz bir dilde yazdırır.
fread()
fseek()    İmleci ileri ve geri almak için kullanılır.
```

```
frewind()  İmleci en başa taşır.
ftell()    Konum Göstergecinin byte olarak nerede olduğunu gösterir.
```

❗ fopen()'de dosya yolu belirlerken çift tırnak içinde backslash kullanamayacağımız için çift backslash ile işlem yaparız.

```
fopen("C:\\Desktop\\DevSaves\\New_Folder","r");
```

Fonksiyon Kontrolü

- Fonksiyonların çalışıp çalışmadığını kontrol etmek için if() içinde kontrol edilmesi gerekir. fopen() fonksiyonu örneği aşağıdadır.

```
if(fopen() == NULL){
    printf("Islem Basarisiz");
    return 1;
}
else{
    printf("Islem Basarili");
}
```

Modlar:

```
r  okuma modu
w  yazma veya oluşturma modu
a  sonuna ekleme ve oluşturma modu (append'den geliyor muhtemelen.)
r+ okuma ve yazma (Dosya mevcut olmalı.)
w+ okuma, yazma ve dosya oluşturma (önceki veriler silinir.)
a+ okuma, ekleme ve dosya oluşturma (önceki verilerin sonuna eklenir.)
rb Dosyadan veri okumak için dosyayı binary olarak aç.
wb Dosyaya veri yazmak için dosyayı binary olarak aç.
```

```
ab Dosyaya veri eklemek için dosyayı binary olarak aç.
rb+ Dosyaya veri yazmak veya okumak için binary olarak aç.
wb+ Dosyaya veri yazmak, oluşturmak veya okumak için binary olarak aç.
ab+ Dosyaya veri eklemek veya okumak için binary olarak aç.
```

Örnek:

```
FILE *ic_dosya_adi; //file tipinde bir işaretçi tanımlandı.
ic_dosya_adi=fopen("dis_dosya_adi","dosya_acma_modu");

ÖRNEK:
FILE *fp; //file pointer tanımlandı.
fp=fopen(file_name,"mode); //file_name yerine aynı yerde değillerse dosya yol
u yazılır.
if(fp==NULL){
    puts("Dosya Acilamadi\n");
    return 1;
}
fclose(fp);
```

Fonksiyonlar

fgetc() & fgets() & feof() Fonksiyonu

```
fgetc(icdosyaadi);

FILE *dosya;
dosya=fopen("font.txt","r");
char ch;
ch=fgetc(dosya) //İlk karakteri okur.
ch=fgetc(dosya) //İkinci karakteri okur.

DÖNGÜ OLUŞTURMA
```

```
do{
   ch=fgetc(dosya);
   printf("%c",ch);
}while(ch!=EOF);
```

- getc() fonksiyonu her seferinde bir karakter ilerleyerek okur, eğer sona geldiyse EOF (end of file) olur. Yukarıdaki do-while döngüsü ile yazdırma işlemi EOF'e ulaşılana kadar devam eder.
    - Bunun yerine aşağıdaki feof() fonksiyonu da kullanılabilir:

```
do{
   ch=fgetc(dosya);
   printf("%c",ch);
}while(!feof(dosya));  //feof olmadığı sürece devam eder. Eşitlik koyamaya ger
ek yoktur.
```

```
char data[100];
fgets(dizi,eleman sayisi,ptr);
fgets(data,100,ptr);
```

- gets() fonksiyonu sadece tek satır okuyabilir, tamamını okumak için döngü içinde yazılması gerekir.

```
while(fgets(data,100,ptr)){
   fprintf("%s",data);
}
```

fscanf() Fonksiyonu

- Formatlı okumak için kullanılır.
- sscanf() ile syntax'ı aynıdır.

```
fscanf(fp, "%d %f %s", &age, &height, name);
```

fputc() & fputs() Fonksiyonu

```
fputc('M',ptr);
fputs("Hello World",ptr);
```

- Değişken yazdırırken:

```
char *name = "Hello World";
fputs(name,ptr);
```

fprintf() Fonksiyonu

```
int i=10;
fprintf(ptr,"Hello World %d",i);
```

fwrite() Fonksiyonu

```
fwrite(dizi,sizeof(dizi_tipi),eleman_sayisi,ptr);
```

- fwrite ile yazılan yazılar okunamaz, okuyabilmek için yine fread() fonksiyonu kullanılmalıdır.
- fwrite() kullanılırken fopen() fonksiyonu yazma modunda açılması gerekir (w, wb etc.)

fread() Fonksiyonu

- fread() kullanılırken fopen() fonksiyonu okuma modlarından birinde açılması gerekir. (r,rb,r+ etc.)

  ```
  fread(dizi,sizeof(int),7,ptr);
  ```

- fread() ve fwrite() fonksiyonlarının syntax'ları aynıdır.

ftell() & fseek & frewind() Fonksiyonu

- ftell() fonksiyonu işaretçinin bulunduğu konumu atar.

```
ftell(ptr);
fseek(ptr,5,SEEK_SET);   //5 birim ileri öteler.
```

1. SEEK_CUR: Mevcut konuma göre değiştir.
2. SEEK_SET: Başlangıçtan itibaren değiştirir.
3. SEEK_END: Sondan itibaren değiştirir.